

Hamza Qureshi - Roll No. 04

Dr. Azam

Advance Analysis of algorithm

24 November 2024

Assignment 03

Divide & Conquer Vs Dynamic Programming

Divide and Conquer:

Divide and Conquer is an approach where a problem is divided into smaller sub problems those sub problems solved independently and then combined the results to get final solution.

- It is basically divide-first, solve-later approach.

Steps Involve:

Divide: Break the problem into smaller sub problems.

Conquer: Solve each sub problem recursively.

Combine: Merge the results of the sub problems to form the solution to the original problem.

- 🚩 But the drawback of this approach I can think of is that the problem that is one time solved may appear again as a sub problem in recursive call means you need to solve same problem more than 2-times.

Example code:

Merge Sort use Divide and conquer to sort an array

```
function merge_sort(arr) {  
  if (arr.length < 2) {  
    return arr;  
  }  
}
```

```
const MidIndex = Math.floor(arr.length / 2); //slice array into half
```

```

const leftArr = arr.slice(0, MidIndex);
const rightArr = arr.slice(MidIndex, arr.length);

return merge(merge_sort(leftArr), merge_sort(rightArr)); // merge is O(n) and merge_sort is //
O(log n)
}
// compare elements of leftArr with rightArr
function merge(leftArr, rightArr) {
  let resultArr = [];
  let leftIndex = 0;
  let rightIndex = 0;

  while (leftIndex < leftArr.length && rightIndex < rightArr.length) {
    if (leftArr[leftIndex] < rightArr[rightIndex]) {
      resultArr.push(leftArr[leftIndex]);
      leftIndex += 1;
    } else {
      resultArr.push(rightArr[rightIndex]);
      rightIndex += 1;
    }
  }
  return resultArr
    .concat(leftArr.slice(leftIndex))
    .concat(rightArr.slice(rightIndex));
}

const arr8 = [1, 1, 34, 2, 4, 9, 6, 99, 6.4];
console.info(merge_sort(arr8));

```

Dynamic Programming:

Dynamic Programming is another technique used to solve complex problems by breaking the problem into sub problems and storing the results (**memoization**) of these sub problems to avoid redundant computations.

It provides key features:

1. **Overlapping Sub problems**
2. **Optimal Substructure**

- ✓ Now the same problem doesn't need to be solved multiple times. It leverages the previous computations to optimize overlapping sub-problems.

There are two Approaches in DP:

1. **Memoization (Top Down Approach):**
2. **Tabulation (Bottom Up Approach)**

1. **Memoization** use recursion and stores results of sub problems in a **cache** to avoid redundant calculations.
 - On demand: Values are computed and stored only when needed.
2. **Tabulation** Builds a table from the base case up to the desired solution, filling values iteratively.
 - Full table is constructed before returning the result.

Memoized Version:

```
function fibonacci(n, memo = {}) {  
  if (n <= 1) return n;  
  
  // Check if result is already computed  
  if (n in memo) return memo[n];
```

```
// Store result in memo for reuse
memo[n] = fibonacci(n - 1, memo) + fibonacci(n - 2, memo);
return memo[n];
}
console.log(fibonacci(10)); // Output: 55
```

Tabulation Version:

```
function fibSeries(n,) {
  const table = Array(n + 1).fill(0);
  table[1] = 1; // kind of base case

  for (let i = 0; i <= n; i++) {
    table[i + 1] += table[i];
    table[i + 2] += table[i];
  }
  return table[n];
}
console.info(fibSeries(50));
```